

Apuntes del CPRE Foundation Level Handbook

Contents

1 Principios fundamentales e ideas básicas

El handbook establece nueve principios fundamentales que aplican a las tareas y prácticas de Requirements Engineering. Estos principios son la base metodológica para elicitar, documentar, validar y gestionar requisitos [?, p. 16].

1. **Orientación al valor.** Los requisitos son un medio para obtener resultados, no un fin documental en sí mismo.
2. **Stakeholders.** RE busca satisfacer deseos y necesidades de las partes interesadas.
3. **Entendimiento compartido.** Sin base común entre actores, el desarrollo falla.
4. **Contexto.** El sistema no se entiende aislado de su entorno.
5. **Problema-requisito-solución.** Es una tríada fuertemente entrelazada.
6. **Validación.** Un requisito no validado no aporta valor real.
7. **Evolución.** El cambio de requisitos es normal, no excepcional.
8. **Innovación.** Repetir lo mismo no basta en muchos contextos.
9. **Trabajo sistemático y disciplinado.** Es imprescindible para mantener calidad y control.

Como idea básica transversal, el documento insiste en balancear coste y beneficio: la inversión en RE reduce riesgo de retrabajo costoso en fases tardías [?, pp. 16–18].

2 Definición y tipos de requisitos

El handbook define **requisito** desde tres puntos de vista complementarios: como una necesidad percibida por un stakeholder, como una capacidad o propiedad que el sistema debe tener y como la representación documentada de esa necesidad o capacidad [?, p. 10].

Con esa base, el documento insiste en que los requisitos existen incluso cuando no se han documentado, y que forman la base de cualquier desarrollo o evolución de sistemas. También diferencia tres clases principales: requisitos funcionales, requisitos de calidad y restricciones [?, p. 10].

El propio handbook amplía el marco conceptual con la definición de Requirements Engineering como un enfoque sistemático y disciplinado para especificar y gestionar requisitos con el objetivo de entender los deseos y necesidades de los stakeholders y reducir el riesgo de entregar un sistema inadecuado [?, pp. 10–11]. A la vez, define stakeholder como cualquier persona u organización que influye en los requisitos o se ve afectada por el sistema [?, p. 11].

3 Vocabulario y conceptos clave

Una parte esencial del enfoque CPRE es trabajar con terminología precisa y compartida. El handbook define conceptos base que deben utilizarse de forma consistente en toda especificación [?, pp. 10–11].

- **Requirement:** necesidad percibida, capacidad del sistema o representación documentada de ambas.
- **Requirements specification:** colección sistemática de requisitos que cumple criterios de calidad.
- **Requirements Engineering:** enfoque sistemático y disciplinado para especificar y gestionar requisitos.
- **Stakeholder:** persona u organización que influye en requisitos o es impactada por el sistema.
- **System / context / scope:** delimitaciones clave para entender qué se diseña, qué se asume y qué queda fuera.

El capítulo de glosarios del documento refuerza que la terminología acordada debe mantenerse de forma centralizada, con uso obligatorio en los artefactos del proyecto para evitar ambigüedades y malentendidos [?, pp. 72–73].

4 Función del ingeniero de requisitos

El rol de *Requirements Engineer* no siempre coincide con un cargo formal, pero el handbook define con claridad su función. Una persona actúa en este rol cuando elicitá, documenta, valida y/o gestiona requisitos como parte de sus responsabilidades, y cuando tiene criterio para definir procesos RE y seleccionar prácticas adecuadas [?, p. 14].

Las responsabilidades clave son:

- tender puentes entre el problema del negocio y las posibles soluciones técnicas;
- promover entendimiento compartido entre stakeholders y equipo técnico;
- aplicar prácticas de calidad sobre requisitos y sus cambios.

El handbook también remarca que este rol puede estar distribuido entre perfiles como analistas de negocio, product owners, especialistas de aplicación, ingenieros de sistemas o desarrolladores, y que requiere habilidades blandas (escucha, negociación, mediación y empatía) además de capacidades técnicas [?, pp. 14–15].

5 Criterios de calidad y contenido de un requisito

Un requisito de calidad debe aportar valor, estar ligado a un stakeholder concreto y situarse en un contexto claro. El handbook subraya que escribir requisitos no es un fin en sí mismo: su valor depende del beneficio que aportan frente al coste de especificarlos, validarlos, documentarlos y gestionarlos [?, pp. 16–17].

Además, un requisito debe considerar las perspectivas de los stakeholders, porque RE trata de satisfacer sus deseos y necesidades, no solo las del usuario final. Eso obliga a identificar roles, priorizar su influencia y resolver conflictos entre puntos de vista distintos [?, pp. 18–19].

También debe quedar claro el contexto del sistema, su límite y su alcance. El handbook explica que los requisitos nunca aparecen aislados, sino dentro de un entorno relevante para

entender el sistema; por eso hay que delimitar qué está dentro y fuera de alcance y qué supuestos del contexto deben cumplirse [?, pp. 22–24].

Por último, un requisito debe ser explícito, verificable y validado. El documento insiste en que la validación comprueba que las necesidades quedan cubiertas y que existe acuerdo entre las partes interesadas. En la gestión posterior, además, conviene registrar identificación, prioridad, dependencias, origen, racionalidad y tipo del requisito [?, pp. 109–111, 136–137].

6 Formalización del requisito

El handbook presenta la formalización como un proceso iterativo que va desde una necesidad implícita hasta un requisito explícito, entendible, revisable y gestionable. Una secuencia práctica de pasos es la siguiente:

1. **Identificar la necesidad.** El punto de partida es una necesidad, deseo o problema percibido por un stakeholder [?, p. 10].
2. **Elicitar la información.** Hay que extraer las necesidades ocultas en sus fuentes y convertirlas en requisitos explícitos usando técnicas de recopilación, colaboración, observación o análisis de artefactos [?, pp. 88–95].
3. **Explorar ideas y alternativas.** Cuando el problema admite soluciones innovadoras, se usan técnicas de diseño e ideación como brainstorming, prototipos o escenarios [?, pp. 96–99].
4. **Resolver conflictos.** Las necesidades de diferentes stakeholders pueden chocar; antes de avanzar, deben identificarse, analizarse y resolverse mediante consenso, compromiso, votación, overruling o definición de variantes [?, pp. 100–108].
5. **Documentar con la representación adecuada.** El requisito puede expresarse como texto natural, plantilla o modelo. El handbook explica que un work product puede ir desde una simple frase hasta una especificación completa de requisitos, y que la forma elegida depende del nivel de abstracción y del propósito [?, pp. 30–32].
6. **Validar.** Antes de entregar el requisito, se debe comprobar que refleja las necesidades reales y que los stakeholders están de acuerdo; la validación debe empezar dentro de RE, no al final del proyecto [?, pp. 109–111].
7. **Gestionar ciclo de vida, versiones y cambios.** Una vez formalizado, el requisito debe quedar bajo control de versiones, trazabilidad y gestión del cambio para mantener coherencia entre estados, configuraciones y baselines [?, pp. 131–140].

7 Ciclo de vida completo de los requisitos

El handbook trata los requisitos como entidades vivas que evolucionan durante desarrollo y operación. En su ciclo de vida, un requisito puede crearse, elaborarse, validarse, consolidarse, implementarse, usarse, cambiarse, mantenerse, archivarse o retirarse [?, pp. 130–132].

Fases y controles más relevantes a considerar:

1. **Creación y elaboración.** Elicitación, análisis y documentación inicial.
2. **Validación y consolidación.** Comprobación de cobertura, acuerdo y calidad.
3. **Implementación y uso.** El requisito pasa de especificado a operado en producto.
4. **Cambio y mantenimiento.** Evolución controlada mediante gestión de cambios.

5. **Retiro, archivo o eliminación.** Cierre del requisito según su vigencia.

Para gestionar este ciclo de vida, CPRE enfatiza: estados y transiciones explícitas, control de versiones, configuraciones y baselines, atributos de requisito y trazabilidad para evaluar impacto de cambios [?, pp. 131–140].

7.1 Estados y transiciones del requisito

Un requisito transita por los siguientes estados a lo largo de su vida:

1. **Creado (Draft)** — Requisito inicial, documentación preliminar.
2. **Elaborado** — Detallado, analizado, listo para validación.
3. **Validado** — Aprobado por stakeholders relevantes.
4. **Consolidado** — Incluido en una baseline de referencia.
5. **Implementado** — Código desarrollado según el requisito.
6. **En producción** — Sistema operativo con el requisito activo.
7. **Mantenido** — Sujeto a cambios controlados durante operación.
8. **Finalizado** — Eliminado de la especificación activa.
9. **Archivado** — Almacenado para referencia histórica.

Observaciones importantes:

- Los cambios pueden generar ciclos: desde *En producción*, un requisito puede pasar a *Mantenido* para cambios controlados y volver a *En producción*.
- La transición de *Validado* a *Finalizado* también es posible si se rechaza un requisito antes de implementación.
- No todos los requisitos alcanzan *En producción*; algunos pueden descartarse o cambiar de estado según prioridad.
- Cada transición debe ser documentada: qué triggeró el cambio, cuándo, y qué impacto tuvo.

8 Plantilla de registro de requisito alineada con CPRE

Tomando como base la documentación oficial usada en el handbook (incluyendo los atributos recomendados por ISO 29148 en la sección de gestión), esta plantilla define un formato mínimo común para registrar requisitos de forma uniforme [?, pp. 30–32, 136–137].

Plantilla recomendada

Campo	Contenido esperado
ID único	Identificador inmutable (ej.: REQ-SEG-001).
Nombre corto	Título sintético y trazable.
Tipo	Funcional, calidad o restricción.
Descripción formal	Enunciado verificable, no ambiguo.
Fuente	Stakeholder, documento, sistema legado, norma, etc.
Rationale	Por qué existe y qué objetivo cubre.
Prioridad (stakeholder)	Alta/media/baja o escala acordada.
Riesgo	Impacto/probabilidad asociados al incumplimiento.
Dependencias	Requisitos relacionados o prerequisites.
Dificultad estimada	Esfuerzo relativo para implementación.
Criterio de validación	Cómo se comprobará su cumplimiento.
Estado de ciclo de vida	Borrador, validado, implementado, archivado, etc.
Versión	Número de versión y fecha.
Trazabilidad	Enlaces backward/forward (origen, diseño, prueba, entrega).

Ejemplo de registro

Campo	Valor de ejemplo
ID único	REQ-PAGO-012
Nombre corto	Confirmación de pago con evidencia
Tipo	Funcional
Descripción formal	El sistema deberá generar una confirmación de pago con identificador de transacción y marca temporal en menos de 5 segundos tras autorización exitosa.
Fuente	Entrevista con Responsable de Finanzas (2026-04-10)
Rationale	Reducir reclamaciones y facilitar auditoría de cobros
Prioridad	Alta
Riesgo	Alto (impacto legal y operativo)
Dependencias	REQ-SEG-004, REQ-INT-007
Dificultad	Media
Criterio de validación	Prueba E2E con 100 transacciones; 100% con comprobante y $t < 5 s$
Estado	Validado
Versión	1.1 (2026-04-11)
Trazabilidad	Caso de prueba TC-PAGO-021; historia US-45; módulo pagos/api-confirm

9 Herramientas comunes en industria para requisitos

- **Jira (Atlassian):** gestión de backlog, historias, priorización y seguimiento del trabajo del equipo.

- **Confluence (Atlassian):** documentación colaborativa de requisitos, decisiones y criterios de aceptación.
- **Azure DevOps (Boards + Wiki):** planificación, seguimiento de requisitos e integración con el desarrollo.
- **Miro / FigJam:** apoyo visual para talleres, lluvia de ideas, refinamiento y alineación temprana.

Nota de uso

En práctica, suelen combinarse una herramienta de backlog o documentación y otra de colaboración visual; en proyectos más exigentes pueden añadir herramientas especializadas de trazabilidad.